Architecture des Systèmes d'Exploitation : Gestion, Cycle de Vie et Communication des Processus sous Unix/Linux

Thème principal : Gestion des processus, filiation (fork/exec), cycle de vie à 3 états et signaux POSIX

Synthèse exécutive : Ce cours pose les fondements du fonctionnement interne d'un système d'exploitation Unix/Linux en analysant le concept clé de processus. Il explique la transition critique entre un fichier exécutable statique (stocké sur disque) et une instance dynamique en cours d'exécution (occupant la RAM et le processeur). Pour permettre l'exécution simultanée de plusieurs programmes sur un matériel limité, le noyau s'appuie sur le multitâche préemptif et ordonnance les processus à travers trois états fondamentaux (Élu, Prêt, Bloqué). Le cours détaille également la structure de contrôle PCB, la généalogie des processus remontant au processus initial systemd/init (PID 1), le mécanisme de création d'instances par les appels système fork() et exec(), ainsi que le contrôle et la communication asynchrone via la gestion des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie et Attributs

Lorsqu'un programme informatique est simplement enregistré sur le disque dur, il s'agit d'une entité passive consommant uniquement de l'espace de stockage. Pour devenir actif, le système d'exploitation doit le charger en mémoire vive (RAM) et lui attribuer des ressources système : c'est la naissance d'un processus. Afin de gérer efficacement ces instances, le noyau Linux associe à chaque processus une structure d'information appelée PCB (Process Control Block). Le PCB contient l'identifiant unique du processus (PID), l'identifiant de son parent (PPID), le statut des registres processeur, la plage d'adresses mémoire allouée et la table des fichiers ouverts. Comme le nombre de processus dépasse largement le nombre de cœurs de processeur disponibles, le système recourt au multitâche préemptif. L'ordonnanceur du noyau fait basculer les processus selon un modèle strict à trois états : 1. Élu (Running) : Le processus est directement pris en charge par un cœur de processeur et exécute ses instructions. 2. Prêt (Ready) : Le processus dispose de toutes les ressources nécessaires pour s'exécuter, mais attend dans la file d'attente que l'ordonnanceur lui attribue un quantum de temps CPU. 3. Bloqué (Waiting/Blocked) : Le processus est suspendu car il attend un événement externe (une saisie au clavier, une lecture de fichier sur disque dur ou une réponse réseau). Pendant ce temps, il ne consomme aucun cycle CPU. Remarque fondamentale : Lorsqu'une opération d'E/S se termine pour un processus Bloqué, le noyau le fait repasser à l'état Prêt (et jamais directement à l'état Élu), afin de respecter l'équité de la file d'attente de l'ordonnanceur.

- Programme vs Processus : Un programme est un ensemble d'instructions passif rangé sur disque. Un processus est l'exécution active de ce code en RAM.
- PCB (Process Control Block) : Structure de données interne au noyau Linux contenant toutes les métadonnées d'un processus (PID, PPID, UID, état, registres CPU, descripteurs de fichiers).
- Multitâche Préemptif : Découpage du temps processeur en très courtes tranches (quanta de temps) pour donner l'illusion d'une exécution simultanée sur un seul cœur.
- Modèle à 3 États : Élu (occupe le CPU), Prêt (en file d'attente CPU), Bloqué (en attente d'un événement externe ou d'une E/S).

Cas concret enseignant : *L'analogie du livre de recettes de cuisine donnée par l'enseignant : Le programme stocké sur disque dur est assimilé à un livre de recettes rangé dans une bibliothèque (passif, il ne consomme ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, occupant de la place en RAM sur son plan de travail, consommant des ressources et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

2. Filiation, Arborescence et Mécanismes de Création (fork et exec)

Sous Unix et Linux, aucun processus ne peut apparaître spontanément ex nihilo. L'ensemble des processus forme une arborescence généalogique stricte. Lors du démarrage du système (boot), le noyau initialise le premier

processus espace utilisateur, historiquement appelé 'init' et désormais nommé 'systemd' sur la majorité des distributions Linux modernes. Ce processus fondateur porte toujours le PID 1. Lorsqu'un utilisateur tape une commande dans un shell (par exemple 'ls' ou 'vim'), la création de ce nouveau processus s'effectue obligatoirement en deux phases distinctes via des appels système : 1. fork() : Le processus parent (le shell) demande au noyau de se dupliquer. Le noyau crée un processus enfant qui est une copie conforme du parent (mêmes variables, même contexte d'exécution), mais qui se voit attribuer son propre PID distinct. 2. exec() (ou une de ses variantes comme execve) : Le processus enfant fait appel à exec() pour remplacer son code en mémoire par le binaire du nouveau programme à exécuter. L'espace mémoire est réinitialisé avec les nouvelles instructions, mais l'identifiant PID reste inchangé.

- Arborescence des processus : Structure hiérarchique où chaque processus a un unique parent (PPID), remontant à la racine systemd/init (PID 1).
- Appel système fork() : Duplique le processus appelant à l'identique pour créer un processus enfant.
- Appel système exec() : Remplace l'image mémoire du processus courant par un nouveau programme binaire à exécuter.

3. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme la plus élémentaire de communication inter-processus (IPC) sous Unix. Un signal est un avertissement court (représenté par un numéro et un nom standardisé) envoyé de façon asynchrone à un processus pour lui notifier un événement. Lorsqu'un processus reçoit un signal, trois comportements sont possibles : - Exécuter l'action par défaut définie par le système (généralement la terminaison du processus). - Ignorer le signal. - Capturer le signal au moyen d'un gestionnaire personnalisé (signal handler) programmé en C/C++. Cependant, pour préserver le contrôle absolu de l'administrateur système sur la machine, deux signaux majeurs font exception et NE PEUVENT PAS être interceptés ni ignorés : SIGKILL (signal 9) et SIGSTOP (signal 19). Ces signaux sont traités directement par le noyau Linux sans jamais consulter le code du processus destinataire.

- Signal POSIX : Interruption logicielle asynchrone envoyée par le noyau ou un processus vers un autre processus.
- Signal Handler (Gestionnaire) : Fonction spécifique définie par le développeur pour capturer et personnaliser la réaction à un signal.
- SIGINT (Signal 2) : Interruption utilisateur envoyée via la combinaison de touches Ctrl + C.
- SIGTERM (Signal 15) : Signal de demande d'arrêt propre et contrôlé, envoyé par défaut par la commande kill.
- SIGKILL (Signal 9) : Ordre d'arrêt immédiat et inconditionnel directement exécuté par le noyau (incapturable).
- SIGSTOP (Signal 19) et SIGCONT (Signal 18) : Mettre en pause et reprendre l'exécution d'un processus.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble donnée par l'enseignant : Le signal SIGTERM (15) est comparé au gardien d'immeuble qui frappe poliment à la porte pour vous demander de quitter les lieux (vous avez le temps d'enregistrer vos fichiers, de nettoyer votre bureau et de fermer proprement). Le signal SIGKILL (9) représente les forces de sécurité qui défoncent la porte et vous expulsent instantanément sans vous laisser le temps de toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Pour la commande 'kill', les options identifiant le signal doivent impérativement être précédées d'un tiret (ex: -9 ou -SIGKILL). Si vous écrivez 'kill 9 1234', Linux interprète '9' non pas comme le numéro de signal, mais comme un premier PID de processus à détruire, puis '1234' comme un second PID. La commande tente d'envoyer le signal par défaut (SIGTERM 15) aux processus 9 et 1234. Si le processus PID 9 n'existe pas ou ne vous appartient pas, le terminal renvoie une erreur de type 'No such process' ou 'Operation not permitted'.

Question étudiant : Qu'est-ce qu'un processus 'Zombie' (defunct) et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie est une tâche qui a terminé son exécution (via exit), mais dont l'entrée subsiste dans la table des processus car son parent n'a pas encore lu son code de retour via l'appel système wait(). Un zombie ne consomme ni RAM ni processeur, mais occupe un numéro de PID. On NE PEUT PAS détruire un Zombie directement avec 'kill -9' car le processus est DÉJÀ mort ! Pour le faire disparaître, il faut soit forcer le père à lire son état, soit tuer le processus parent : le zombie devient alors orphelin, est adopté par systemd/init (PID 1) qui nettoie immédiatement sa ligne dans la table des processus.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur système exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le mécanisme déclenché lors de la commande kill -15 et indiquez l'état du processus lors d'une vérification ultérieure.

Démarche de résolution :
Étape 1 : Le symbole '&' placé en fin de ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche immédiatement le PID attribué au processus et rend la main à l'utilisateur sans bloquer l'invite de commande. Étape 2 : La commande 'ps -ef | grep sleep' filtre l'ensemble des processus du système pour afficher la ligne correspondant au binaire 'sleep'. Cela permet de lire son PID (ex: 4582) ainsi que son PPID (le PID du shell Bash). Étape 3 : La commande 'kill -15 4582' transmet le signal SIGTERM (Signal 15) au processus 4582. Le processus reçoit l'instruction de s'arrêter proprement, ferme ses descripteurs et libère sa mémoire.

```
#!/bin/bash
# Séquence de démonstration de la gestion des processus sous Linux

# 1. Lancement d'une tâche de 600 secondes en arrière-plan
sleep 600 &

# 2. Recherche du PID du processus sleep dans la table système
ps -ef | grep sleep

# 3. Envoi du signal SIGTERM ( signal 15 ) pour demander un arrêt propre
kill -15 4582
```

Résultat : Le processus sleep capte le signal SIGTERM, s'arrête de façon contrôlée et est immédiatement retiré de la table des processus. Une nouvelle commande 'ps -ef | grep sleep' ne renvoie plus le processus sleep.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Il s'agit du processus systemd (ou init sur les systèmes anciens), et son PID est impérativement 1.

Q : Quel signal POSIX est émis lorsqu'un utilisateur saisit le raccourci clavier 'Ctrl + C' dans un terminal ?

R : Il s'agit du signal SIGINT (Signal 2), qui demande l'interruption du processus s'exécutant au premier plan.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un programme développé par un utilisateur ?

R : Parce que SIGKILL n'est pas transmis au processus lui-même : il est intercepté et exécuté directement par le noyau Linux pour garantir à l'administrateur le moyen de stopper n'importe quel processus de manière inconditionnelle.

Q : Lorsqu'un processus passe de l'état Bloqué à la suite de la fin d'une lecture disque, vers quel état le noyau le bascule-t-il ?

R : Le noyau le bascule vers l'état Prêt (Ready), et non directement à l'état Élu, afin qu'il repasse par la file d'attente de l'ordonnanceur.